

DistilBERT & ALBERT

DistilBERT is a distilled version of BERT meaning it is trained using knowledge distillation a technique where a smaller model (student) learns from a larger model (teacher). It retains 97% of BERT's performance while being 40% smaller and 60% faster making it highly efficient for NLP tasks such as text classification, sentiment analysis and question-answering.

The Goal of DistilBERT

The primary goal of DistilBERT is model compression and efficiency. The researchers at Hugging Face wanted to create a smaller, faster, and cheaper version of BERT that could be deployed in environments with limited computational resources (like on mobile phones or in large-scale production APIs) while retaining as much of BERT's impressive performance as possible.

Their aims were to:

Reduce Model Size: Make a model with significantly fewer parameters.

Increase Speed: Make the model much faster during both training and inference.

Reduce Cost: Lower the memory and computational power required to run the model.

Minimize Performance Drop: Achieve all of the above with only a minor decrease in performance compared to the full BERT model.

Key Architectural Differences

DistilBERT simplifies BERT's architecture while maintaining performance:

DistilBERT does not have token-type embeddings which reduces its model complexity hence more lightweight. But in the process, it has lost the ability to differentiate sentence relationships which means that it cannot perform well in tasks like next-sentence prediction.

The reduction of layers from 12 to 6 maintains the balance between depth and efficiency in DistilBERT. With this architectural change, training is much faster.

The model also uses dynamic masking during training which enables it to work best with different kinds of texts. While this is taking place, it helps DistilBERT to learn sturdy representations of words, hence improving the way it carries itself over a wide range of NLP tasks.

How DistilBERT Works: Knowledge Distillation

DistilBERT works through a process called Knowledge Distillation, a teacher-student framework designed for model compression.

Example: Imagine a highly experienced chef (BERT) teaching a junior chef (DistilBERT). Instead of memorizing recipes, the junior chef learns key cooking techniques, allowing them to prepare similar dishes faster with fewer ingredients.

The Setup: A large, pre-trained teacher model (BERT-base) is used to teach a smaller student model (DistilBERT) with a reduced architecture (6 layers instead of 12).

The Learning Signal: The student isn't just trained on correct answers ("hard labels") but learns to mimic the teacher's full output probability distributions ("soft labels"). This

transfers the teacher's "dark knowledge," like its understanding of similarities between words.

The Loss Function: The training combines three objectives:

Distillation Loss – Encourages the student model to mimic the teacher's soft predictions.

Language Modeling Loss – Predicts the next word in a sentence.

Cosine-Distance Loss – Aligns the hidden state representations of the student and teacher models.

Distillation Loss (L_{ce}): Distillation loss is essential in training the student model during knowledge distillation. It's calculated by comparing the student's prediction probability to the soft probabilities provided by the teacher model.

Masked Language Modeling Loss (L_{mlm}): Like BERT, DistilBERT learns language patterns by a technique called masked language modeling (MLM). In MLM certain words in a sentence are randomly replaced with a special token, and the model learns to predict these masked words based on the surrounding context.

Cosine Embedding Loss (L_{cos}): Cosine embedding loss is another important component used in the DistilBERT training process. Its primary purpose is to ensure that the student model learns to replicate the teacher model's internal knowledge representations ([embeddings](#)) by minimizing the cosine distance between the embeddings generated by the student and teacher models.

Triple loss

Finally, the loss function in DistilBERT is determined by adding the linear combinations of the three loss functions. Based on the loss value, the student model undergoes backpropagation to update its weights.

$$\text{Loss} = L_{mlm} + L_{ce} + L_{cos}$$

The temperature T controls the smoothness of the probability distribution:

When $T = 1$, this is equivalent to the standard softmax.

When $T > 1$, the output probabilities are smoother.

When $T < 1$, the output probability distribution becomes more rough.

In other words, the Higher the temperature, the smoother the probability distribution.

Advantages of DistilBERT

Speed and Efficiency: With fewer parameters (66 million vs. BERT's 110 million), DistilBERT is faster to train and deploy, making it ideal for resource-constrained settings.

Scalability: Its smaller footprint allows it to scale across edge devices, democratizing access to advanced NLP.

Performance: Despite its size, DistilBERT delivers near-BERT-level accuracy, making it a practical choice without sacrificing too much quality.

Limitations of DistilBERT

While DistilBERT is impressive, it's not without trade-offs. The reduction in size means it may struggle with extremely complex language tasks where BERT's deeper architecture excels.

For cutting-edge research or niche applications requiring peak performance, the original BERT or even larger models like RoBERTa might still be preferred.

Performance Trade-off Inherent, small accuracy drop compared to BERT-base.

Reduced Capacity Fewer parameters and layers limit its ability to store complex knowledge.

Inherited Knowledge Only knows what its teacher model knew; cannot learn new facts during distillation.

ALBERT is a parameter-efficient version of BERT. Its primary goal is to reduce memory consumption and allow for larger models through architectural changes (parameter sharing and factorized embeddings), often leading to better performance on benchmarks than BERT, especially on knowledge-intensive tasks.

The Goal of ALBERT

The primary goal of ALBERT is to significantly improve parameter efficiency and enable the scaling of pre-trained models to larger sizes without a corresponding increase in memory consumption and training time. Developed by researchers at Google and Toyota Technological, ALBERT was designed to address the hardware limitations that hinder the training of ever-larger BERT-like models, allowing for more sustainable and accessible research and deployment.

Its aims were to:

Drastically Reduce Parameter Count: Achieve a dramatic decrease in the number of model parameters through innovative architectural changes.

Reduce Memory Usage and Training Cost: Lower the GPU/TPU memory footprint during training, enabling the training of larger models on existing hardware. **Maintain or Improve**

Performance: Ensure these efficiency gains do not come at the cost of capability, often aiming to surpass BERT's performance on benchmarks. **Facilitate Model Scaling:** Provide a sustainable architecture that makes it feasible to build and experiment with models of much larger capacity.

Key Architectural Differences

The backbone of ALBERT architecture is similar to BERT that is encoder layers with GELU activation function. However, there are three main changes that are present in ALBERT but not in BERT.

Factorization of the Embedding matrix:

In the BERT model and its improvements such as XLNet and RoBERTa, the input layer embeddings and hidden layer embeddings have the same size. But in this model, the authors separated the two embedding matrices. This is because input-level embedding needs to refine only context-independent learning but hidden level embedding requires context-dependent learning. This step leads to a reduction in parameters by 80% with a minor drop in performance when compared to BERT.

Cross-layer parameter sharing:

The authors of this model also proposed the parameter sharing between different layers of the model to improve efficiency and decrease redundancy. The paper proposed that since the previous versions of BERT, XLNet, and RoBERTa have encoder layer stacked on top of one another causes the model to learn similar operations on different layers. The authors proposed three types of parameter sharing in this paper:

- Only share Feed Forward network parameter

- Only share attention parameters

- Share all parameters. Default setting used by authors unless stated otherwise.

Inter Sentence Coherence Prediction:

Similar to the BERT, ALBERT also used Masked Language model in training. However, Instead of using NSP (Next Sentence Prediction) loss, ALBERT used a new loss called SOP (Sentence Order Prediction). NSP is a binary classification loss for predicting whether two segments appear consecutively in the original text, the disadvantage of this loss is that it checks for coherence as well as the topic to identify the next sentence. However, the SOP only looks for sentence coherence.

How ALBERT Works: Parameter Sharing, Factorization & Inter-Sentence Coherence

ALBERT works through three main ideas: Factorized Embeddings, Cross-Layer Parameter Sharing, and Inter-Sentence Coherence Prediction. Together they drastically cut down the number of parameters while improving how the model understands relationships between sentences.

Example:

Imagine a library (BERT) with 12 separate rooms, each with its own full set of books (parameters). ALBERT is like a library where all the rooms share the same set of books and only have smaller copies of the covers. You still get all the knowledge but with fewer resources. And instead of just knowing which books are next to each other, it also learns how the chapters flow coherently.

The Setup :

ALBERT keeps the same Transformer encoder architecture as BERT but introduces:

Smaller embeddings: Input words are first mapped to a low-dimensional embedding space (128 dimensions instead of 768).

Projection layer: These small embeddings are then projected to the hidden size used by the Transformer (768).

Shared layers: All Transformer layers reuse the same parameters instead of each layer having its own.

This results in a much smaller model (ALBERT-base $\approx$ 12M parameters vs. BERT-base $\approx$ 110M) but with the same depth.

The Learning Signal

Like BERT, ALBERT still learns language understanding through Masked Language Modeling (MLM), but it replaces BERT's Next Sentence Prediction (NSP) with a stronger signal called Inter-Sentence Coherence Prediction (ISCP) — also known as Sentence Order Prediction (SOP):

Masked Language Modeling (MLM): Randomly masks words and predicts them using context.

Inter-Sentence Coherence Prediction (ISCP/SOP): Given two consecutive segments from the same document, predict whether they are in the correct order or swapped.

Because both segments come from the same text, the model can't cheat by spotting topic differences; it must learn true coherence between sentences.

The Training Objectives

ALBERT's loss combines:

MLM Loss (L_{mlm}): Predict masked tokens.

ISCP/SOP Loss (L_{sop}): Predict correct or swapped sentence order.

Loss= $L_{mlm}+L_{sop}$ ALBERT learns directly through these two objectives.

Factorized Embedding Parameterization

In BERT, token embeddings have the same dimension as the hidden size, making the embedding matrix huge.

ALBERT factorizes it:

Embedding matrix (small): Maps tokens $\rightarrow$ small dimension.

Projection matrix: Expands small dimension $\rightarrow$ hidden size.

This cuts down parameters dramatically without hurting performance.

Cross-Layer Parameter Sharing

Instead of each Transformer layer having its own weights, ALBERT reuses the same parameters for:

Attention weights

Feedforward network weights

This means the model depth is preserved (12 layers) but parameter count is tiny.

Advantages of ALBERT

Much fewer parameters: Lower memory footprint and faster training/inference.

Keeps model depth Even with shared parameters : you still have the same number of layers (12 or 24), which preserves representational power.

Better inter-sentence understanding : Replacing NSP with SOP (Inter-Sentence Coherence Prediction) gives stronger signals for tasks like QA, NLI, and reading comprehension.

Reduced overfitting risk Parameter sharing acts like a form of regularization.

Limitations of ALBERT

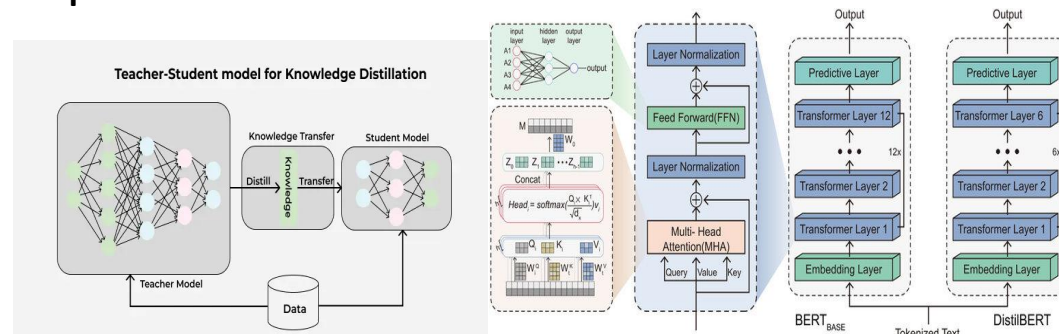
Limited capacity for very large-scale tasks : While parameter sharing reduces size, it also caps how much information can be stored uniquely per layer some very large tasks may benefit from separate weights per layer as in BERT.

Training time can still be long : Despite fewer parameters, ALBERT's depth and weight sharing still require substantial compute for pretraining.

Parameter sharing may reduce layer diversity : All layers having the same weights can limit the variety of transformations learned at different depths.

Not inherently faster per step : Inference can still take time because the model is deep (just smaller in memory).

Graphs & tables



Models	MNLI	QNLI	QQP	RTE	SST	MRPC	CoLA	STS
BERT-large	86.6	92.3	91.3	70.4	93.2	88.0	60.6	90.0
XLNet-large	89.8	93.9	91.8	83.8	95.6	89.2	63.6	91.8
RoBERTa-large	90.2	94.7	92.2	86.6	96.4	90.9	68.0	92.4
ALBERT (1M)	90.4	95.2	92.0	88.1	96.8	90.2	68.7	92.7
ALBERT (1.5M)	90.8	95.3	92.2	89.2	96.9	90.9	71.4	93.0

Model	IMDb (acc.)	SQuAD (EM/F1)
BERT-base	93.46	81.2/88.5
DistilBERT	92.82	77.7/85.8
DistilBERT (D)	-	79.1/86.9

References

[Introduction to DistilBERT Model - GeeksforGeeks](#)

[DistilBERT in Natural Language Processing - GeeksforGeeks](#)

[Distilbert: A Smaller, Faster, and Distilled BERT - Zilliz Learn](#)

[mjdgcandcagmikhbjnilkmfnjeamfikk/https://arxiv.org/pdf/1910.01108](https://arxiv.org/pdf/1910.01108)

[ALBERT - A Light BERT for Supervised Learning - GeeksforGeeks](#)

[Large Language Models, ALBERT - A Lite BERT for Self-supervised Learning | Towards Data Science](#)

[extension://mjdgcandcagmikhbjnilkmfnjeamfikk/https://arxiv.org/pdf/1909.11942](https://arxiv.org/pdf/1909.11942)

[ALBERT: Efficient NLP with Lite BERT Architecture](#)